Phase 1 — Data Collection and Tokenizer Construction

Two Independent Monolingual Transformer Language Models
Model H (Hindi) and Model L (Nepali)

Yogendra Patel | Roll No. 2025201098

CL3410 — Language Models and Agents, Monsoon 2026
International Institute of Information Technology, Hyderabad

19 August 2026

Code repository	github.com/CL3-410/individual-project-CReativeRebe1
Dataset (Google Drive)	drive.google.com/drive/folders/1lkP2GJKqgr-uzzz-WLeChOM_flqOe8xQ
Dataset (HuggingFace)	huggingface.co/datasets/YogendraPatel/LMA_Individual_Project

Headline result. Two entirely independent corpora were built from scratch: **Hindi 517,640,653 tokens** across 3,587,045 documents (22.5 % manually collected) and **Nepali 572,506,257 tokens** across 5,536,928 documents (24.7 % manually collected). Both exceed the ∼500M-token target and both clear the ≥20 % manual-collection floor. Each language has its own cleaning pipeline with independently calibrated thresholds, its own Unigram-16k tokenizer, and its own encoded training arrays. Zero documents are shared between the two corpora, verified mechanically. The full test suite is 354 tests.

1 Language Selection and Project Constraints

The assignment requires two monolingual decoder-only Transformer language models of roughly 25M parameters each: one for a higher-resource language (**Model H**) and one for a lower-resource language (**Model L**), with the explicit constraint that the two models share *no* data, tokenizer, vocabulary, or weights.

Hindi was chosen as Model H and **Nepali** as Model L. The pairing is deliberate rather than convenient, and it makes the project harder in a way that is useful:

- **Same script, different languages.** Both are written in Devanagari (Unicode U+0900–U+097F). This removes the trivial separation that a script difference would provide: a Hindi document and a Nepali document cannot be told apart by looking at the character set, so language identification has to do real work. Section 3.4 quantifies exactly how often an off-the-shelf classifier gets this wrong.
- **Genuine resource asymmetry.** Hindi has orders of magnitude more crawlable web text than Nepali, and public language-identification and NLP tooling has seen far more Hindi. This is the asymmetry the assignment is about, and it shows up as measurable differences throughout this report — not as a vague claim.
- **Shared vocabulary, divergent orthography.** Both draw heavily on Sanskrit, so their vocabularies genuinely overlap; but their orthographic conventions differ in ways that matter for normalisation (Section 3.3.1). A normalisation rule that is correct for Hindi *corrupts* Nepali, which is the single clearest illustration in this project of why the two pipelines had to be separated.

Independence discipline. Throughout, the two languages share *code* but never *artifacts*. The shared library `lma/` holds only language-neutral machinery: MinHash and LSH, union-find, n-gram decontamination, corpus statistics, plotting, and the SentencePiece wrapper. Everything that encodes a fact about Hindi or Nepali — stopword lists, boilerplate phrases, normalisation rules, every threshold — lives in `hindi/clean/` or `nepali/clean/`, which are separate Python packages with separate configuration objects. Section 7 verifies this mechanically rather than asserting it.

2 Data Collection

2.1 Sources and the two-sided corpus design

The corpus has two sides, and the distinction between them is load-bearing for the assignment's ≥20 % manual-collection requirement.

Manual side (collected by this project). Three independent collection routes:

1. **Asynchronous polite crawler** (`scripts/run_crawl.py`) — an `aiohttp` crawler with per-host concurrency limits, `robots.txt` and `Crawl-delay` compliance, a SQLite-backed resumable frontier in WAL mode, and round-robin fairness across domains so no single fast host monopolises the fetch budget.
2. **Fast crawler** (`scripts/fast_crawl.py`) — a higher-throughput variant used once the frontier was well populated with known-good hosts.
3. **Wikipedia dumps** (`scripts/run_wiki.py`) — streamed directly from the compressed Wikimedia dumps with `bz2` + `ElementTree.iterparse` + `mwparserfromhell`, rather than crawled. This is both faster and kinder to Wikimedia's servers, and it yields the *original* article text rather than machine-translated mirrors.

Downloaded side (public corpus). `ai4bharat/IndicCorpV2`, pulled by `scripts/fetch_downloaded.py` in HuggingFace streaming mode: 600M words per language.

Table 1: Raw collection, before any cleaning. Word counts are whitespace tokens.

Side	Source	Documents	Words
Hindi (Model H)			
Manual	Crawler	68,446	27,671,716
Manual	Fast crawler	103,134	79,001,007
Manual	Wikipedia dump	116,889	51,939,070
Manual	**Total**	**288,469**	**158,611,793**
Downloaded	IndicCorpV2	6,399,541	600,000,011
Nepali (Model L)			
Manual	Crawler	27,568	17,150,211
Manual	Fast crawler	312,653	141,665,551
Manual	Wikipedia dump	30,580	8,502,324
Manual	**Total**	**370,801**	**167,318,086**
Downloaded	IndicCorpV2	10,119,070	600,000,086

The composition difference drives everything downstream. Wikipedia is 33 % of Hindi's manual words but only 5 % of Nepali's; Nepali's manual side is 85 % news fast-crawl. This single fact explains why the two languages needed different quality thresholds (Section 3.3), why their survival rates differ by 20 percentage points (Section 4), and why their sentence-level duplication behaves oppositely (Section 4.3).

2.2 Deduplicating the downloaded side against the manual side

The downloaded pull is deduplicated against the manual corpus *at fetch time*, and the direction of that deduplication is a deliberate decision:

> **Duplicates are always dropped from the downloaded side, never the manual side.**

Manual tokens are the scarce, graded resource — the $\geq$20 % floor is measured against them, and more of them can only be obtained by crawling for hours. Downloaded tokens are effectively unlimited: if one is discarded, the next one is free. Accordingly the 600M target counts only what is actually *written*, so deduplication losses are absorbed by streaming further rather than by silently shrinking the corpus.

Two checks run per streamed document, cheapest first: an exact SHA-256 over normalised text, then a MinHash/LSH near-duplicate lookup against an index built over the entire manual corpus (288,469 documents for Hindi). Normalisation is applied to the *hash key only* — the stored document keeps its original bytes, because matching is a comparison decision, not a licence to rewrite what gets stored. No cleaning of any kind is applied to the downloaded side: IndicCorpV2 is already a cleaned corpus, and re-filtering it would both waste hours and blur the manual-versus-downloaded comparison this report has to make.

3 The Cleaning Pipeline

3.1 Architecture: two pipelines, one shared mathematics library

The cleaning cascade implements all fourteen steps of the pipeline described in the Yi technical report (arXiv:2403.04652, Figure 1), plus a decontamination stage taken from Lecture 3. It is implemented **once per language**. Figure 1 shows the complete data flow.

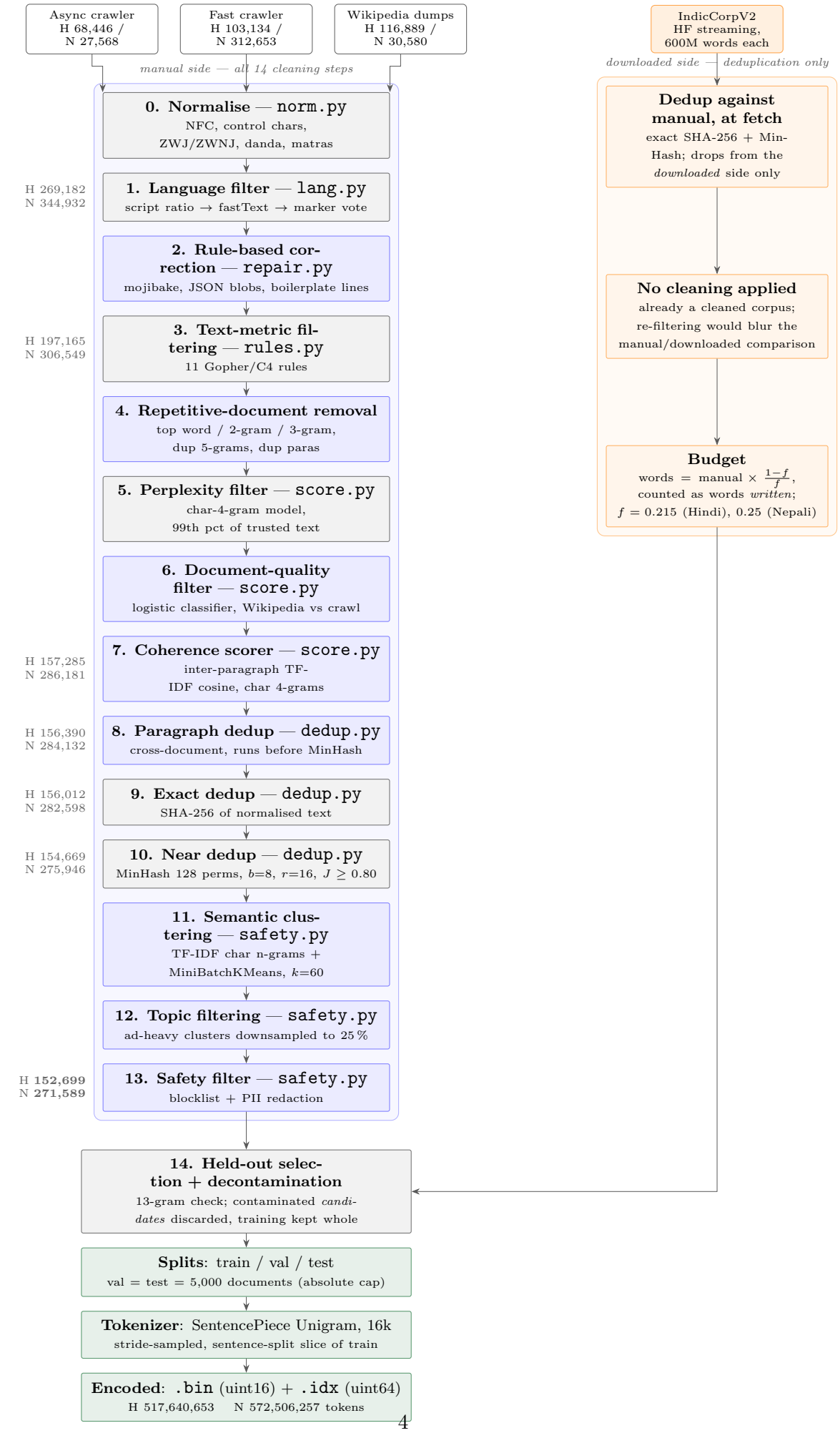


Figure 1: The complete Phase-1 data flow. The manually collected side passes through all fourteen cleaning

3.2 Three deliberate deviations from Yi's ordering, each made on measurement

(i) Correction runs *before* filtering. Yi's figure places rule-based correction after the metric filters. In that order, a news article carrying three furniture lines and a leaked JSON metadata blob is *rejected* before anything gets the chance to repair it. Repairing first and filtering second recovers real prose. On Hindi, repair fired on roughly 117,000 documents: 22,867 JSON blobs, 31,671 boilerplate lines, 25,100 lines containing a boilerplate phrase, 36,683 runs of repeated punctuation, and 607 mojibake sequences.

A safety valve prevents this from becoming a laundering step: if repair would remove more than 50 % of a document's *characters*, the original is returned untouched, so a document that is mostly boilerplate still fails the filters honestly and the funnel attributes it to the correct stage. That valve was originally written in terms of *lines* and had to be changed: crawled articles typically arrive as one long prose line plus several short furniture lines, so a line-based valve read a 95 %-prose document as 60 % boilerplate and refused to repair it.

(ii) Coherence uses character 4-grams, not words. The coherence scorer measures mean TF-IDF cosine similarity between consecutive paragraphs, to catch documents stitched together from unrelated fragments. Word-level cosine barely separated a coherent Devanagari article from a deliberately stitched one: **0.031 versus 0.029**. Both languages are inflected, so consecutive paragraphs of one article share *stems*, not surface forms, and exact word matching misses the overlap being measured. Character 4-grams separated the same pair **2× better: 0.052 versus 0.024**.

The IDF weighting also had to be corrected. The textbook form $\mathrm{idf}(t) = \log\big(N/(1+\mathrm{df}(t))\big)$ is *exactly zero* when a term appears in 2 of 3 paragraphs — that is, it zeroes out precisely the shared terms the metric exists to detect. Every document scored 0.0000. The smoothed form $\mathrm{idf}(t) = \log\big((N+1)/(1+\mathrm{df}(t))\big) + 1$ fixes this.

Short documents *bypass* the check and pass by default: with fewer than three paragraphs or 150 words, the cosine is undefined-to-noisy and a genuinely fine three-sentence news blurb scores near zero purely for lack of vocabulary overlap. On a low-resource corpus an unmeasurable score must never become a rejection.

(iii) Decontamination cleans the held-out side, not the training side. See Section 4.3; this is the single largest design decision in the pipeline and is given its own section.

3.3 Per-language divergence: the same rule, different numbers

Lecture 3 (slide 30) warns that "a threshold tuned on English will empty your corpus" and advises setting thresholds by reading what they remove, in your own language. The same argument applies between two Devanagari languages with different corpus compositions. Every divergence below was measured, not guessed, and each is recorded with its measurement in the relevant `clean/config.py`.

Table 2: Thresholds that differ between the two pipelines, with the measurement behind each.

Threshold	Hindi	Nepali	Evidence
fastText LID minimum confidence	0.65	**0.50**	lid.176 labels 95.9 %/95.2 % of Nepali wiki/crawl correctly, but with median confidence only 0.76/0.69 — right and unsure. Hindi's 0.65 bar rejects **17.9 %** of correctly-labelled Nepali Wikipedia and 22.5 % of crawl; 0.50 rejects 3.0 % and 0.7 %.
Quality-classifier minimum	0.30	**0.10**	Held-out means: Hindi wiki 0.841 vs crawl 0.390; Nepali wiki 0.766 vs crawl 0.177. Keep rates (wiki/crawl) for Nepali: 0.30 → 87.0/19.2 %, 0.20 → 92.4/26.8 %, **0.10 → 96.8/49.4 %**. Hindi's 0.30 keeps 94/55 %, so 0.10 is the Nepali setting at the same operating point. Keeping 0.30 would discard 81 % of the Nepali crawl — four-fifths of that corpus.
Perplexity cutoff	16.68	18.01	99th percentile of each language's own trusted Wikipedia text.
Coherence minimum	0.01234	0.01061	1st percentile of trusted text. The first hand-picked value, 0.04, was measured to sit at Wikipedia's *10th* percentile — it would have rejected ∼9 % of Wikipedia itself.
Chandrabindu folding	yes	**no**	See Section 3.3.1.
Manual-fraction target	0.215	0.25	Hindi's manual side cleans down harder (52.9 % vs 73.2 % survival), and the downloaded budget is derived from the manual word count, so a lower target was needed to reach the token goal.

3.3.1 Normalisation: where a correct Hindi rule corrupts Nepali

Both pipelines share the script-level normalisation in `lma/text/normalize.py`: NFC composition, control-character stripping, zero-width joiner/non-joiner cleanup, danda normalisation, whitespace collapsing, and optional digit conversion. Idempotency ($\text{norm}(\text{norm}(x)) = \text{norm}(x)$) is enforced by test.

They diverge on one rule with real consequences. **Hindi folds chandrabindu (U+0901) to anusvara (U+0902)**: modern Hindi typesetting treats the two as interchangeable for nasalisation, and folding reduces vocabulary fragmentation at no semantic cost. **Nepali must not.** In Nepali the chandrabindu is standard spelling and carries the nasal vowel in extremely common words — *gaum* (village, U+0917 U+0916 U+0901), *thaum* (place), *humda* (when being), *baki* (remaining). Folding it would rewrite these as misspelt Nepali, fragmenting the tokenizer's vocabulary against every correctly-spelled occurrence elsewhere in the corpus, and would put wrong spellings into a model intended to generate the language.

Both pipelines additionally repair encoding corruption that is not spelling: collapsing repeated matras, dropping word-initial dependent vowel signs (a matra cannot legally begin a word), and removing dangling viramas left by broken HTML extraction. Nukta is preserved in Hindi, since U+095B (*za*) and similar characters distinguish Perso-Arabic loanwords that are genuinely frequent.

3.4 Language identification, measured out-of-sample

An earlier version of this figure was built *in-sample*: the character-n-gram classifier was evaluated on the same seed documents it had been fitted on, so its diagonal dominance was guaranteed and

told the reader nothing. It was rebuilt (`scripts/langid_report.py`) against labels that come from outside the pipeline entirely — the Wikipedia dump a document came from — read from the **raw** shards, before any language filter runs. fastText's `lid.176` is a pretrained model this project never trained, so this is genuinely held-out for it. Evaluating on the cleaned splits instead would have been circular in a subtler way than the original bug: those documents survived a fastText check, so fastText would have been scored against its own decisions.

Table 3: Language identification on held-out Wikipedia-dump text (3,000 documents per language; labels from the dump source, not from the pipeline).

	Documents	fastText correct	Labelled as the *other* language	Marker vote correct
Hindi	3,000	**99.97 %**	0	97.30 %
Nepali	3,000	**95.87 %**	**119**	**99.97 %**

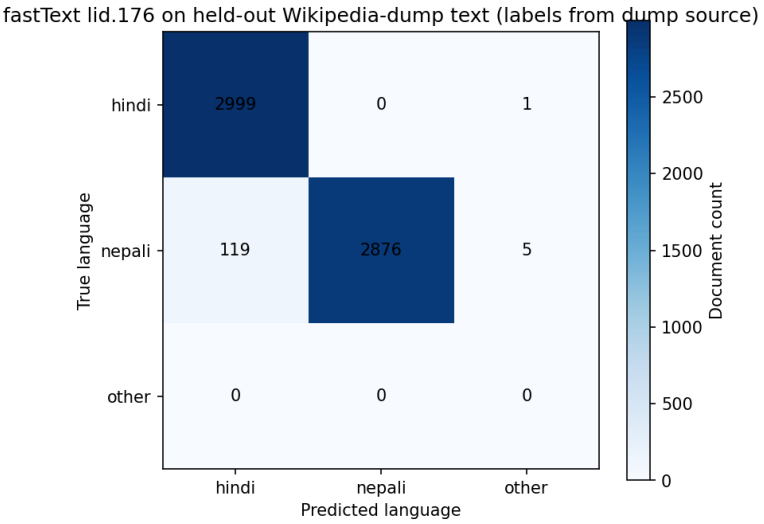


Figure 2: Confusion matrix for fastText lid.176 on held-out Wikipedia text. The confusion is one-directional.

Two findings, both of which justify design decisions taken elsewhere:

1. **The confusion is one-directional.** fastText labelled 119 of 3,000 Nepali Wikipedia articles as Hindi (4.0 %) and **zero** Hindi articles as Nepali. Nepali is the low-resource side of that model, and its errors fall asymmetrically toward the higher-resource neighbour — exactly the leakage direction that would contaminate the Nepali corpus with Hindi text if fastText were trusted alone. In the real cleaning run, the Nepali pipeline rejected 5,050 documents that fastText labelled `hi` and a further 4,377 on the marker vote.
2. **The marker vote is the better detector on the language that needs it** (99.97 % vs fastText's 95.87 % on Nepali), while fastText is better on Hindi (99.97 % vs 97.30 %). The two are therefore combined rather than ranked: `lang.check` requires the Devanagari script ratio, fastText, and the marker vote to agree before a document is accepted.

4 Results: What the Pipeline Actually Removed

4.1 Cleaning funnel

Table 4: Document survival through the cascade (manual side). Percentages are of documents collected.

Stage	Hindi		Nepali	
	Documents	%	Documents	%
Collected (raw manual)	288,469	100.0	370,801	100.0
After language filtering	269,182	93.3	344,932	93.0
After repair + text-metric rules	197,165	68.3	306,549	82.7
After perplexity / quality / coherence	157,285	54.5	286,181	77.2
After paragraph dedup	156,390	54.2	284,132	76.6
After exact dedup	156,012	54.1	282,598	76.2
After near dedup (MinHash-LSH)	154,669	53.6	275,946	74.4
After topic filtering	152,714	52.9	271,591	73.2
After safety filtering	**152,699**	**52.9**	**271,589**	**73.2**

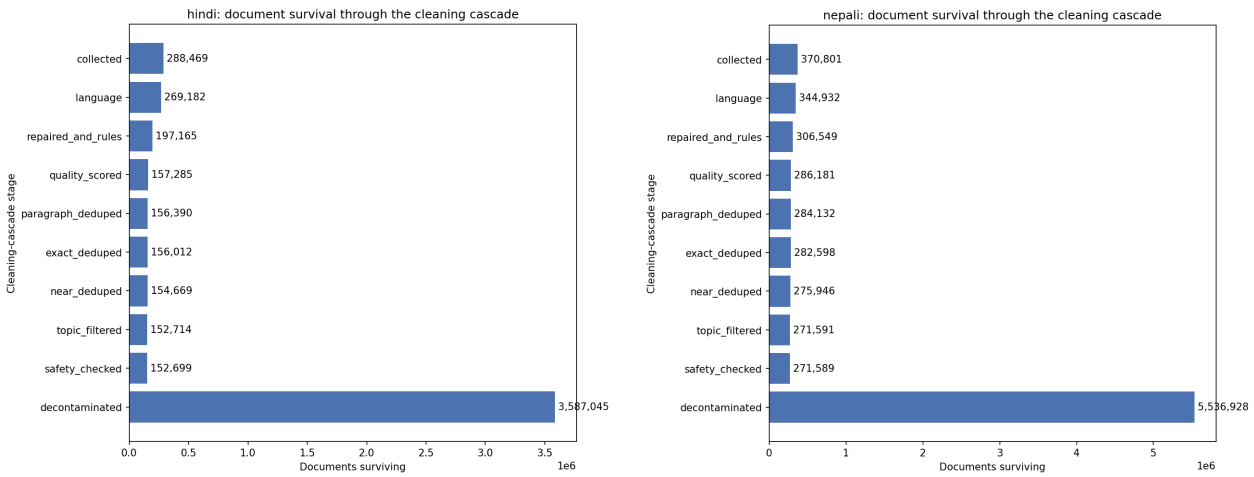


Figure 3: Cleaning funnels. Hindi (left) retains 52.9 % of collected documents; Nepali (right) 73.2 %.

Why Hindi loses 20 percentage points more. The gap is concentrated in the text-metric and learned-quality stages, and it reflects a real difference in what was crawled. Hindi's crawl is dominated by large news portals whose "hub" pages are templates: the duplicate-5-gram fraction has a 90th percentile of **0.88** on the Hindi crawled subset, against **0.405** at the 99th percentile for Nepali. Hindi's crawl contains many more pages that are mostly repeated furniture.

Table 5: Top rejection reasons (documents; a document may violate several rules).

Hindi		Nepali	
`symbol_to_word_ratio_too_high`	41,426	`symbol_to_word_ratio_too_high`	35,431
`quality_too_low`	34,405	`quality_too_low`	18,124
`dup_ngram_fraction`	22,435	`low_devanagari_ratio`	9,765
`dup_paragraph_fraction`	19,079	`lid_low_confidence`	6,650
`low_devanagari_ratio`	18,269	`lid_said_hi`	5,050
`bullet_line_fraction_too_high`	7,662	`hindi_markers_dominant`	4,377

Nepali's distinctive losses are cross-language: 5,050 documents fastText identified as Hindi and

4,377 caught by the marker vote — real Hindi text inside the Nepali crawl, which is precisely what the corpus-independence requirement is about.

4.2 Deduplication

Table 6: What each deduplication stage removed.

	Hindi	Nepali
Paragraphs excised (cross-document)	510,077	1,836,622
from documents	67,059	189,039
Exact duplicate documents	378	1,534
Near-duplicate documents	1,343	6,652
in clusters	1,077	6,584
Ad-heavy clusters downsampled	3 of 60	3 of 60
documents removed	1,955	4,355

Paragraph dedup runs before MinHash, and that ordering matters. Nepali's number is striking: 1,836,622 paragraphs removed across 189,039 documents — more than half of all surviving documents carried at least one paragraph that recurs in three or more other documents, amounting to 17 % of the corpus by bytes. Removing shared footers *first* stops two otherwise unrelated articles from matching each other on boilerplate they both carry. Running MinHash first would have deleted real articles for agreeing about a cookie notice.

The very small exact-duplicate counts (378 and 1,534) confirm that whole-document duplicates had already been handled: the downloaded side was deduplicated against the manual side at fetch time.

Two implementation problems had to be fixed before this stage could run at scale.

1. **MinHash was not reproducible across processes.** The base shingle hash used Python's builtin `hash()`, which `PYTHONHASHSEED` salts per process: the same shingle hashed to 3735431355 in one process and 2056009044 in another. Any multiprocessing deduplication would have silently produced garbage matches, and the `seed=42` field gave false reassurance. Replaced with `xxhash`, chosen on measurement: over 200,000 realistic Devanagari shingles, builtin `hash()` ran at 6.12M/s, `xxhash.xxh64` at 3.25M/s, and `blake2b(digest_size=8)` at only 1.09M/s — so the obvious cryptographic choice would have been 5.6× slower than the broken one, while xxhash costs only 1.9×.
2. **The LSH candidate-pair enumeration would have exhausted memory.** A bucket with k members yields $k(k-1)/2$ pairs, and this corpus provably has that pathology (an early crawl produced 745 byte-identical boilerplate pages out of 950). Replaced with a lazy generator plus a bucket cap that "stars" oversized buckets around a representative: 5,000 identical documents go from ∼12.5M pairs per band to 4,999. Since callers merge verified pairs with union-find, a star still collapses the bucket into one cluster.

4.3 Decontamination: which side pays

Decontamination removes 13-word n-gram overlap between training and held-out data. Without it, Phase 2's perplexity and bits-per-byte on val/test could be inflated by memorised held-out text, with no way for a reader to distinguish a genuinely good score from a leaked one. MinHash cannot substitute for it: MinHash measures *whole-document* similarity, while contamination can be a single shared sentence inside an otherwise-distinct document.

The conventional direction failed badly here. Dropping contaminated *training* documents cost **123,357 of 266,189** Nepali manual training documents — 46.3 % — and pushed the manual share to **15.0 %, below the assignment's 20 % floor**. The cause is a real property of the corpus: the Nepali crawl reuses syndicated agency sentences so heavily that each held-out article collided with roughly 23 training articles. Near-duplicate detection cannot see this, because those documents are ∼0.05 Jaccard-similar, not 0.8 — they are distinct articles that happen to quote the same wire copy.

An intermediate fix (capping the held-out set at 5,000 documents per split) helped far less than expected, and the reason is instructive: the contamination index scales with held-out *length*, not count. The original 110,102-document held-out set was mostly short downloaded documents (∼62 n-grams each); the capped 10,000-document set was 54 % long manual articles (∼370 n-grams each). The index shrank only ∼ 3×, and the training loss moved only from 48.4 % to 46.3 %.

The adopted solution reverses the direction. Since this project constructs *both* sides, and there are millions of candidates available for a 5,000-document held-out set, held-out *candidates* that collide with training text are discarded and replaced from an oversampled pool (2× the cap). Training keeps every document; val and test remain 13-gram-disjoint from train.

Table 7: Decontamination under the adopted (held-out side) direction.

	Hindi	Nepali
Training documents dropped	**0**	**0**
Held-out candidates offered	20,000	20,000
Held-out candidates discarded	3,293 (16.5 %)	8,596 (43.0 %)
Final held-out (val / test)	5,000 / 5,000	5,000 / 5,000
of which manually collected	803 / 771	580 / 552

The cost, stated plainly. This introduces a mild bias: held-out documents are, by construction, ones that do *not* share sentences with training text, so val and test skew slightly toward less-syndicated writing. This is the standard direction for benchmark construction — a benchmark is decontaminated against the training corpus, not the reverse — and it is the affordable direction for a low-resource corpus. The alternative was a corpus that violates the assignment's manual-collection floor.

The candidate-collision rate is itself a finding. 16.5 % for Hindi against 43.0 % for Nepali. Nepali news portals republish agency copy far more heavily. This runs *opposite* to whole-document duplication, where Hindi's crawl is the more duplicated of the two. Hindi has more internally repetitive template pages; Nepali has more distinct articles sharing sentences. They are two different pathologies, which is exactly why both paragraph deduplication and 13-gram decontamination earn their place in the cascade.

5 Final Corpus Statistics

Table 8: Final corpora. Token counts are from the encoder that Phase 2 will read.

	Model H (Hindi)	Model L (Nepali)
Manual words	86,845,726	102,346,029
Downloaded words	322,185,544	312,027,108
Total words	**409,031,270**	**414,373,137**
Manual tokens (text only)	115,682,511	140,233,982
Downloaded tokens (text only)	398,371,097	426,735,347
subtotal	514,053,608	566,969,329
End-of-sequence tokens (one per document)	3,587,045	5,536,928
Total tokens (as encoded)	**517,640,653**	**572,506,257**
Manual share (words)	21.2 %	24.7 %
Manual share (tokens)	**22.5 %**	**24.7 %**
Train documents	3,577,045	5,526,928
Validation documents	5,000	5,000
Test documents	5,000	5,000
Train tokens	515,972,223	571,227,702
Validation tokens	821,762	634,788
Test tokens	846,668	643,767
Distinct source hosts (manual)	9+	8+
Cascade wall-clock	92 min	113 min

Both languages exceed the ∼500M-token target and both clear the ≥20 % manual floor. Note that the manual share is *higher* in tokens (22.5 %) than in words (21.2 %) for Hindi: manually crawled text tokenises slightly less efficiently than IndicCorpV2 text, so the manual side gains share when counted in tokens — which works in favour of the requirement.

The floor is enforced in code, not by inspection. Each pipeline raises at the end of a run if the final manual share falls below 20 % (`MANUAL_FLOOR` in `<lang>/clean/run.py`), after writing `stats.json` so the funnel remains available for diagnosis. This exists because an earlier run produced a 15.0 % corpus and reported success with an ordinary log line that was easy to read past.

Held-out sets are capped in absolute terms. Val and test are 5,000 documents each rather than a percentage. At corpus scale a 1 % fraction meant 110,102 documents and 6.8M n-grams in the contamination index — the cause of the training loss described in Section 4.3. At 5,000 documents, Hindi's validation split is still 821,762 tokens, far more than perplexity or bits-per-byte estimation requires.

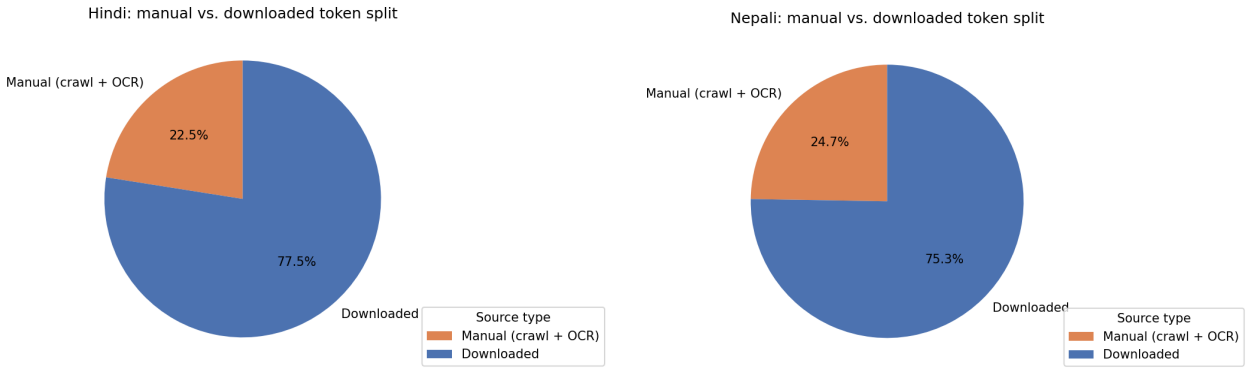


Figure 4: Manual versus downloaded token composition. Hindi (left), Nepali (right).

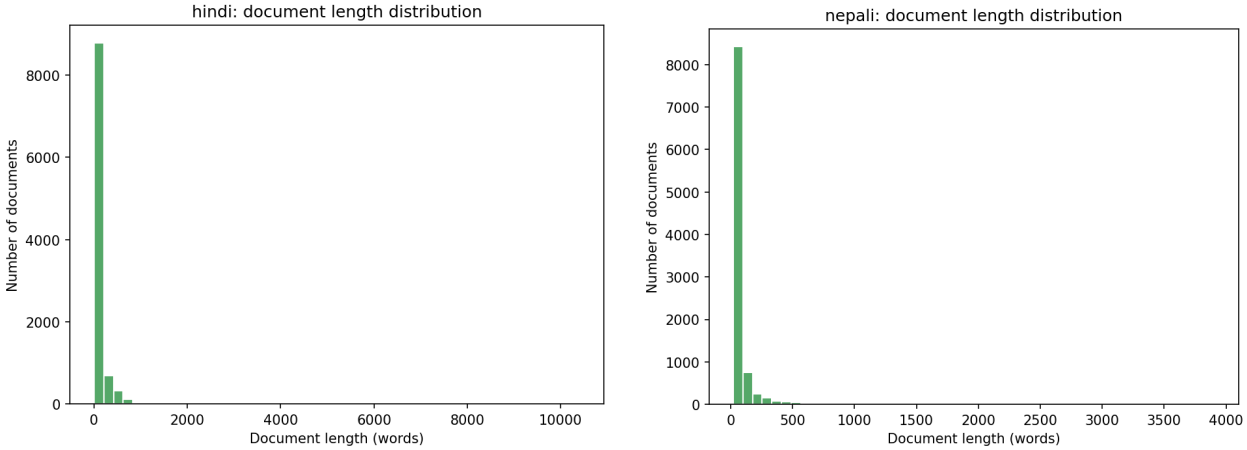


Figure 5: Document length distributions (words per document), drawn from the held-out splits — a uniform 2 % sample of the corpus by construction.

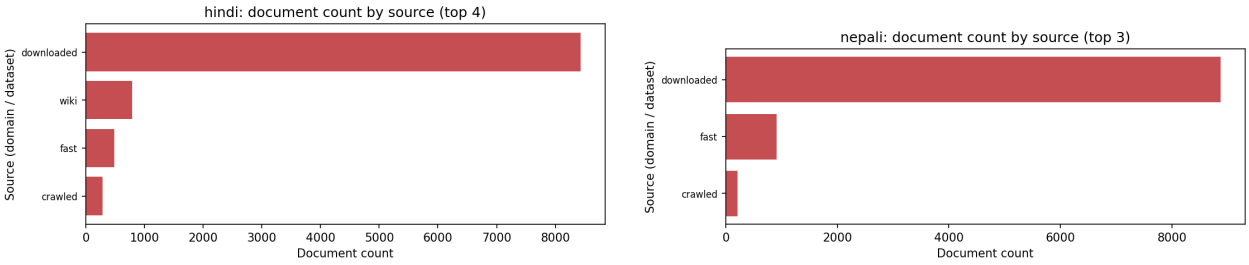


Figure 6: Documents per collection source.

6 Tokenizer Construction

6.1 Sweep design

Two algorithms (SentencePiece BPE and Unigram) × six vocabulary sizes {8k, 12k, 16k, 24k, 32k, 48k} = **12 tokenizers per language, 24 in total**. Each is trained on a sentence-split sample of that language's own training split and evaluated on its real held-out validation split. Vocabularies are never shared between languages.

Fixed settings, chosen before any measurement: `character_coverage=0.9995`, `byte_fallback=True`, and reserved ids `pad=0, bos=1, eos=2, unk=3`.

A silent training bug had to be fixed first. SentencePiece defaults `max_sentence_length` to **4192 bytes** and discards longer lines *without warning*. Devanagari costs 3 bytes per character in UTF-8, so that limit is only ∼1,400 characters, while the corpus splits are written one document per line averaging ∼1.6 kB with a long tail well past the limit. A large share of the corpus was therefore never seen by the trainer. Training input is now split into sentences on danda (U+0964) by `lma/tokenizer/sample.py`, with the limit also raised explicitly as a second line of defence. Documents containing no danda at all (tables, extraction failures) are hard-wrapped, so they cannot be dropped either. Every number below post-dates this fix.

The sample is drawn by *stride* rather than from the head of the file: the splits are written manual-first then downloaded, so a prefix sample of 200,000 documents out of 3.2M would have contained no downloaded text at all, and the tokenizer would have been fitted on a quarter of its own training distribution.

6.2 Sweep results

Table 9: Full sweep, Model H (Hindi). Fertility is tokens per whitespace word on held-out text; lower is better. "Embedding cost" is the share of a 25M-parameter budget consumed by the embedding table at $d_{\text{model}} = 512$ with tied weights.

Vocab	Algorithm	Fertility	Chars/token	UNK rate	Vocab use	Embedding cost
8,000	BPE	1.4211	3.659	0.0 %	0.967	16 %
8,000	Unigram	1.3888	3.742	0.0 %	0.977	16 %
12,000	BPE	1.3390	3.882	0.0 %	0.958	25 %
12,000	Unigram	1.3140	3.955	0.0 %	**0.978**	25 %
16,000	BPE	1.2926	4.021	0.0 %	0.948	33 %
16,000	**Unigram**	**1.2728**	4.084	0.0 %	0.970	33 %
24,000	BPE	1.2420	4.185	0.0 %	0.907	49 %
24,000	Unigram	1.2305	4.224	0.0 %	0.915	49 %
32,000	BPE	1.2155	4.276	0.0 %	0.846	66 %
32,000	Unigram	1.2073	4.305	0.0 %	0.841	66 %
48,000	BPE	1.1877	4.376	0.0 %	0.710	98 %
48,000	Unigram	1.1841	4.389	0.0 %	0.695	98 %

Table 10: Full sweep, Model L (Nepali).

Vocab	Algorithm	Fertility	Chars/token	UNK rate	Vocab use	Embedding cost
8,000	BPE	1.5807	4.116	0.0 %	0.966	16 %
8,000	Unigram	1.5411	4.222	0.0 %	0.972	16 %
12,000	BPE	1.4662	4.438	0.0 %	0.961	25 %
12,000	Unigram	1.4393	4.521	0.0 %	**0.975**	25 %
16,000	BPE	1.4018	4.641	0.0 %	0.958	33 %
16,000	**Unigram**	**1.3802**	4.714	0.0 %	0.970	33 %
24,000	BPE	1.3271	4.903	0.0 %	0.936	49 %
24,000	Unigram	1.3146	4.950	0.0 %	0.931	49 %
32,000	BPE	1.2852	5.063	0.0 %	0.893	66 %
32,000	Unigram	1.2766	5.097	0.0 %	0.877	66 %
48,000	BPE	1.2385	5.254	0.0 %	0.789	98 %
48,000	Unigram	1.2343	5.271	0.0 %	0.762	98 %

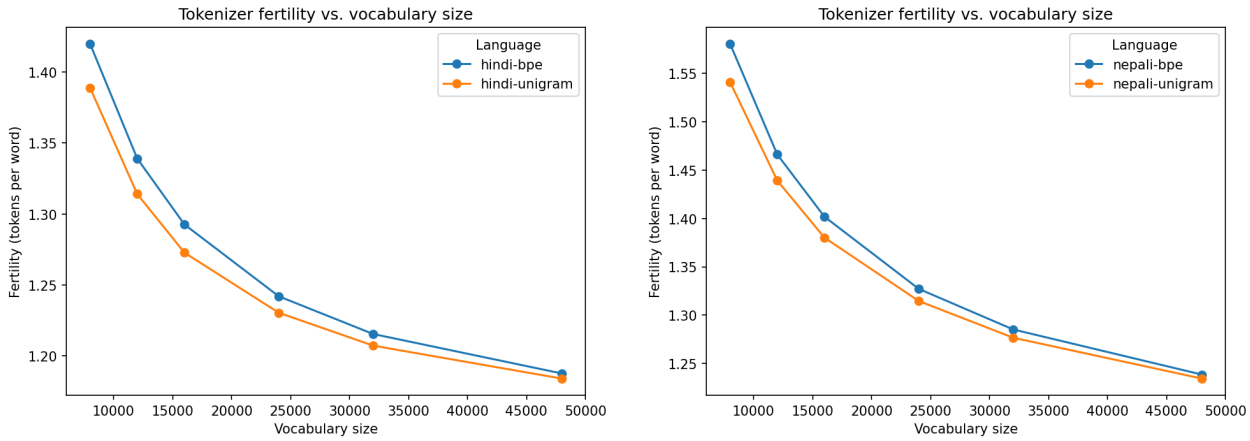


Figure 7: Fertility versus vocabulary size. Both languages show the same shape: steep gains up to ∼16k, then diminishing returns.

Unigram beats BPE at every matched vocabulary size in both languages. That settles the algorithm question on evidence rather than convention. The margin is consistent (0.010–0.040 fertility) and, notably, Unigram also achieves higher vocabulary utilisation at the sizes that matter.

6.3 On the unknown-token rate

The assignment names "fertility / unknown-token rate on held-out text" as the selection criterion. The measured **UNK rate is exactly 0.0 % for all 24 models**. This is not luck: with `byte_fallback=True`, any character the vocabulary lacks decomposes into byte pieces, so no input is unrepresentable *by construction*. This was verified independently rather than trusted from configuration — `tests/test_tokenizer_reference.py` feeds the tokenizer Japanese text and an emoji and asserts the UNK rate is still zero.

Consequently no UNK plot is presented: it would be a flat line at zero for every configuration. The selection criterion collapses to fertility in practice, and this report states that explicitly rather than presenting a 0 % column as though it discriminated between options.

6.4 Selection, and the parameter-budget argument

Selected: SentencePiece Unigram, vocabulary 16,000, for both languages.

Fertility alone always prefers a larger vocabulary. At Phase 2's ∼25M-parameter budget that preference is not free. With tied input/output embeddings, the embedding table costs $V \times d_{\text{model}}$ parameters, while a decoder layer costs approximately $12\,d_{\text{model}}^2$ (attention $4d^2$: W_Q, W_K, W_V, W_O; MLP $8d^2$ at the standard 4× expansion). Vocabulary therefore trades directly against depth:

Table 11: Vocabulary versus affordable depth at a 25M-parameter budget, $d_{\text{model}} = 512$, tied embeddings ($12d^2 = 3{,}145{,}728$ parameters per layer).

Vocab	Embedding params	Share of 25M	Layers affordable	
8,000	4,096,000	16 %	6	
12,000	6,144,000	25 %	6	
16,000	8,192,000	33 %	5	**selected**
24,000	12,288,000	49 %	4	
32,000	16,384,000	66 %	2	
48,000	24,576,000	98 %	**0**	no model is possible

A 48k vocabulary would spend essentially the entire budget on embeddings, leaving room for zero transformer layers — it is not merely expensive, it is unusable. 16k keeps 67 % of the budget

for the model itself, gives up only ∼3 % fertility against 24k, and has higher vocabulary utilisation (0.970 vs 0.915 for Hindi), meaning the larger vocabulary is already going partly unused.

Two assumptions are worth stating, since they are mine rather than the assignment's: $d_{\text{model}} = 512$ (a typical width at this scale, not yet fixed by Phase 2), and tied embeddings. A narrower model changes the arithmetic — at $d_{\text{model}} = 384$ a layer costs 1.77M parameters, so 16k affords 10 layers. Untied embeddings would double the embedding column.

A flaw in automatic selection, worth recording. The sweep's original elbow rule found each algorithm's elbow *independently* and then compared the two at *different* vocabulary sizes. On Nepali, BPE's fertility improvement never fell below the 3 % threshold, so its elbow search ran off the end of the grid and returned 48k; that was then compared against Unigram's elbow at 24k and declared the winner. The rule effectively rewarded whichever algorithm's search ran further. The selection code now compares algorithms at a matched size and warns explicitly when an elbow never fires. The final choice reported here is pinned deliberately and recorded as pinned in `sweep_results.json` rather than presented as an automatic outcome.

6.5 Token-level corpus properties

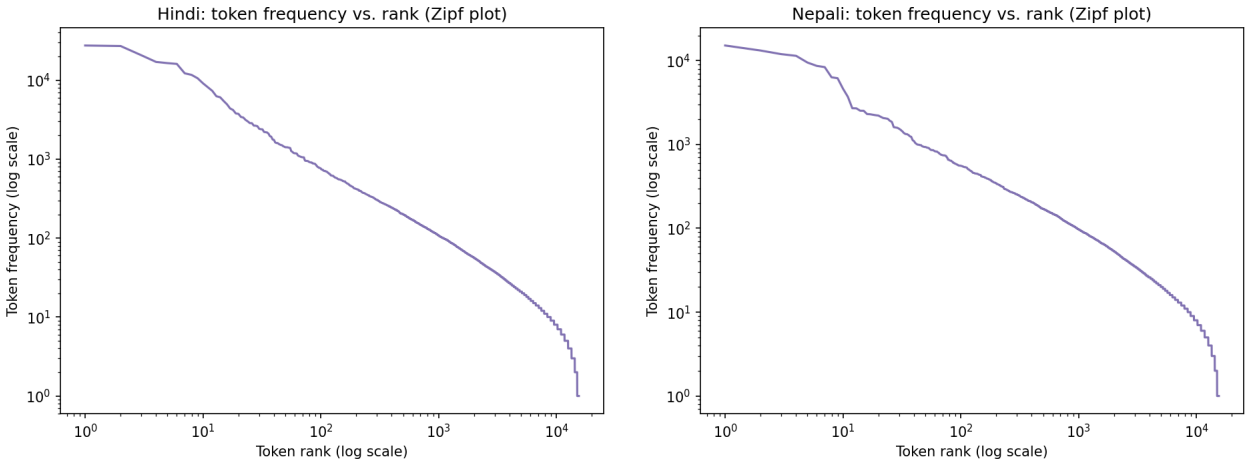


Figure 8: Token frequency versus rank (log–log). Both corpora follow the expected Zipfian distribution, a basic sanity check that the tokenizer is producing a natural-language-like distribution rather than fragmenting text.

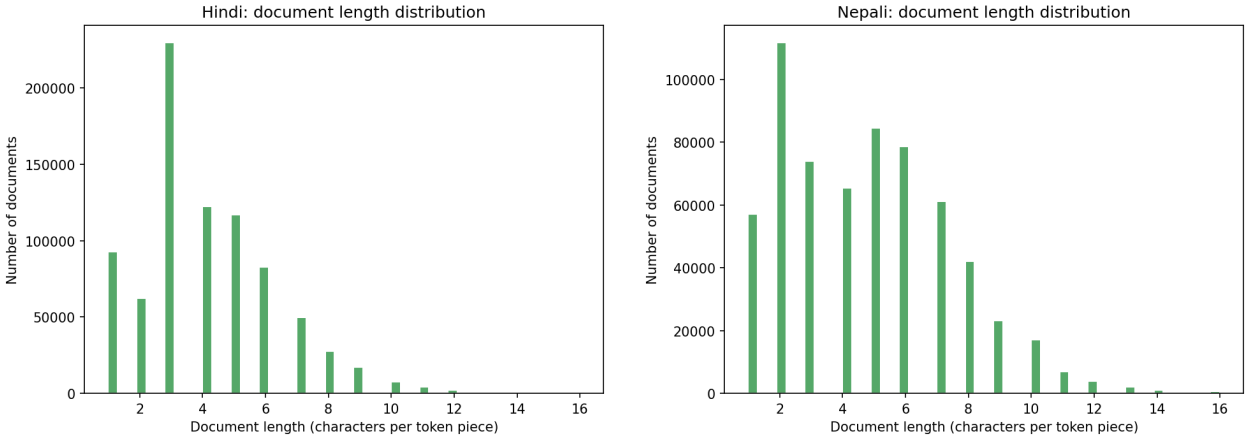


Figure 9: Distribution of token piece lengths in characters.

Nepali's chars-per-token is consistently higher than Hindi's at every vocabulary size (4.714 vs 4.084 at 16k), while its fertility is also higher (1.3802 vs 1.2728). Both follow from Nepali having longer average word forms: more characters per token *and* more tokens per word.

6.6 Qualitative subword tokenization samples and comparative analysis

The metrics above are aggregates. This section looks at what the tokenizers actually *do* to individual sentences, which is where the difference between the two models becomes concrete rather than statistical. Figure 10 takes four real held-out sentences per language — text neither tokenizer was trained on — and segments each one under *both* models, its own and the other language's. Piece boundaries are shown as | and SentencePiece's word-start marker (U+2581) is displayed as _.

Hindi held-out sentences

इन दोनों घटनाओं के पीछे सबसे बड़ा कारण नशा था।

hi 11 _इन|_दोनों|_घटनाओं|_के|_पीछे|_सबसे|_बड़ा|_कारण|_नशा|_था|।

ne 23 _इन|_दो|नों|_घटना|ओं||_के|_पी|छे||_सब|से|_बड|<0xE0>|<0xA4>|<0xBC>|ा|_कारण|_न|शा|_था|।

खुफ़िया ठिकानों पर पुलिस के आला अफ़सरों ने पूछताछ की।

hi 11 _खुफ़िया|_ठिकानों|_पर|_पुलिस|_के|_आला|_अफ़सरों|_ने|_पूछताछ|_की|।

ne 21 _खु|फ|िया|_ठिक|ानों|_पर|_पुलिस|_के|_आ|ला|_अफ|सरों|_ने|_पूछ|ता|छ|_की|।

जल्द ही घटना को वर्क आउट कर दिया जाएगा।

hi 10 _जल्द|_ही|_घटना|_को|_वर्क|_आउट|_कर|_दिया|_जाएगा|।

ne 16 _जल|द|_|ही|_घटना|_को|_वर्क|_आउट|_कर|_द|िया|_जा|ए|गा|।

देर रात तक दस से ज़्यादा लोगों को उठाया गया।

hi 11 _देर|_रात|_तक|_दस|_से|_ज़्यादा|_लोगों|_को|_उठाया|_गया|।

ne 17 _दे|र|_रात|_त|क|_दस|_से|_ज़्यादा|_लोगो||_को|_उठ|।|या|_ग|या|।

Nepali held-out sentences

तर यो नै साझा वा अन्तमि प्रतविेदन होइन।

ne 9 _तर|_यो|_नै|_साझा|_वा|_अन्तमि|_प्रतविेदन|_होइन|।

hi 10 _तर|_यो|_नै|_साझा|_वा|_अन्तमि|_प्रतविेदन|_हो|इन|।

प्रचण्डले शुक्रबारै नयाँ प्रतविेदन पेश गरे।

ne 8 _प्रचण्डले|_शुक्रबारै||_नयाँ|_प्रतविेदन|_पेश|_गरे|।

hi 14 _प्र|च|ण्ड|ले|_शुक्र|बारै||_नयाँ||_प्रतविेदन|_पेश|_ग|रे|।

तर वहिबिार उनले पेश गरेको प्रस्तावबारे नेताहरुले खुलेर बोल्न चाहेनन्।

ne 13 _तर|_वहिबिार|_उनले|_पेश|_गरेको|_प्रस्ताव|बारे|_नेताहरुले|_खुलेर|_बोल्न|_चाहे|नन्|।

hi 25 _तर|_व|हि|बार|_उन|ले|_पेश|_ग|रे|को|_प्रस्ताव|बारे||_नेता|हरु|ले||_खुले|र|_बोल|न|_चाहे|न|न्|।

"बैठकमा हामी झन्डै झन्डै सहमत निजकि पुगेका थियौं।

ne 13 _"|बै|ठ|क|मा|_हामी|_झन्डै|_झन्डै|_सहमत|_नजकि|_पुगेका|_थियौं|।

hi 24 _"|बै|ठ|क|मा|_हा|मी|_झ|न्|डै||_झ|न्|डै||_सहमत|_न|ज|कि|_पु|गे|का|_थ|यौं|।

Figure 10: Qualitative segmentation samples. Blue is the sentence's own-language tokenizer, orange is the other language's. Counts on the left are pieces produced. Generated by `scripts/tokenization_samples.py`; the same data is in `figures/tokenization_samples.json`.

Table 12: Aggregate over the eight sampled sentences: what it costs to tokenize text with the wrong language's model.

Sentences	Words	Own tokenizer	Other tokenizer	Penalty
Hindi (4 sentences)	39	43 tokens (1.103/word)	77 tokens (1.974/word)	**+79.1 %**
Nepali (4 sentences)	32	43 tokens (1.344/word)	73 tokens (2.281/word)	**+69.8 %**

Not one of the eight sentences segments identically under the two tokenizers. Four observations, each visible in the figure:

1. **The own-language tokenizer is close to word-level; the foreign one shatters words.** Hindi's first sample is 10 words: its own tokenizer emits 11 pieces (essentially one per word), while the Nepali tokenizer emits 23 — it has no whole-word entries for this vocabulary and falls back to fragments. The most extreme case is Nepali's *Prachandale* (a political name plus case marker), which the Nepali model stores as a *single* piece and the Hindi model breaks into four (`pra + cha + nda + le`). A name common in Nepali news is simply absent from a vocabulary built on Hindi text.
2. **Byte fallback fires exactly where the orthographies differ.** In Hindi's first sample the word *bara* ("big") carries a nukta (U+093C), the Perso-Arabic diacritic that Hindi uses and Nepali essentially does not. The Nepali tokenizer has no piece containing it, so it decomposes into three raw UTF-8 bytes: `<0xE0>|<0xA4>|<0xBC>`. This is the mechanism behind the 0 % UNK rate made visible — nothing is ever unrepresentable, but representing it costs three tokens instead of being part of one. It is also direct evidence that the vocabularies encode genuinely different orthographic assumptions, not merely different merge orders.
3. **Shared Sanskrit vocabulary shows up as near-agreement.** The Nepali sentence *"tara yo nai sajha wa antim prativedan hoina"* costs 9 pieces under Nepali and 10 under Hindi — almost no penalty, because every content word in it is Sanskrit-derived and common to both languages. This is the same overlap that gives the two vocabularies 5,695 shared pieces (Section 7), seen at the level of a single sentence. The penalty is not uniform: it is small on shared vocabulary and large on language-specific morphology.
4. **Inflectional suffixes are where the models diverge most.** Nepali's agglutinative endings (*-haru*, *-le*, *-ma*, *-bare*) survive as single pieces in the Nepali vocabulary but are re-segmented character-by-character by the Hindi one; the Hindi sample's *-on* plural endings behave symmetrically in reverse. Fertility differences between the two languages (1.2728 vs 1.3802, Section 4) are largely this: Nepali packs more morphology per word, so more pieces per word even under its own tokenizer.

Why this matters beyond the independence check. A 70–79 % token penalty is not a cosmetic difference. Training Model L on text tokenized with Model H's vocabulary would consume roughly 1.7× the context window for the same content, shorten the effective sequence length, and inflate perplexity for reasons that have nothing to do with the model's language ability. This is the practical argument for per-language tokenizers, independent of the assignment requiring them — and it is why fertility, not vocabulary size, was the selection criterion in Section 4.

6.7 Encoding for Phase 2

The selected tokenizer encodes each split to a nanoGPT-style memmap pair: `<split>.bin` holding `uint16` token ids, and `<split>.idx` holding `uint64` cumulative document offsets, so document i occupies `bin[idx[i]:idx[i+1]]`. This lets Phase 2 train over a ∼500M-token corpus without loading it into memory, and lets document boundaries be recovered without re-tokenising.

Table 13: Encoded corpora. Sizes are the `.bin` arrays.

	Hindi	Nepali
Tokenizer	`hindi_unigram_16000.model`	`nepali_unigram_16000.model`
Train tokens	515,972,223	571,227,702
Validation tokens	821,762	634,788
Test tokens	846,668	643,767
Total	**517,640,653**	**572,506,257**
`.bin` size	1.03 GB	1.14 GB
Encoding wall-clock	334 s	459 s

`uint16` is valid precisely because the selected vocabulary (16,000) is far below 2^{16}; the encoder asserts this rather than assuming it. Each split is verified after writing: the arrays are read back and the document count from `.idx` and token count from `.bin` are checked against what the encoder reported. An `.idx` disagreeing with its `.bin` would silently feed Phase 2 documents that start mid-sentence.

A reconciliation worth noting. The encoded totals exceed the tokenizer's own corpus measurement by exactly the document count (Hindi: 514,053,608 + 3,587,045 = 517,640,653; Nepali: 566,969,329 + 5,536,928 = 572,506,257). The difference is the end-of-sequence token appended to each document by the encoder. The two numbers agreeing to the document is a useful consistency check on both.

7 Independence Verification

`scripts/verify_independence.py` checks the assignment's hardest constraint mechanically rather than asserting it in prose. Run against the final corpus, **all checks pass**:

Check	Result
Directory separation	No cross-language path references found
Document overlap	**0** (Hindi 3,587,040 documents vs Nepali 5,536,919; the checker skips blank lines, so these run a handful below the encoder's 3,587,045 / 5,536,928)
Tokenizer files	Model hashes differ (`0683a15aa93d...` vs `371ebc2d5d05...`)
Vocabularies	Both 16,000 pieces; 5,695 pieces shared, 10,305 unique to each
Checkpoints	Skipped — Phase 2 not started

On the 5,695 shared vocabulary pieces. This is stated plainly rather than hidden. Two languages sharing a script and a large Sanskrit-derived lexicon will necessarily share subword pieces; a zero overlap would be evidence of a broken tokenizer, not a virtuous one. What the requirement forbids is a shared tokenizer or shared training data, and both are verified distinct: the models were trained independently on disjoint corpora, and 64 % of each vocabulary is unique to its language.

Behavioural evidence. `scripts/tokenizer_comparison.py` tokenises ten real held-out documents under *both* tokenizers. **Zero of the ten produce identical segmentations**, and the same text costs materially different amounts under each model — 427 vs 668 tokens, 342 vs 563, 1,388 vs 2,333 (means 512 vs 562). The two vocabularies are not merely different files; they segment the same Devanagari differently.

8 Reproducibility

`scripts/reproduce.sh` runs the entire pipeline end to end on a small bundled sample corpus, in about two minutes, with no GPU and no dependency on the live crawl. It operates against an isolated demo directory so it can never read, write, or overwrite the real corpus or the real figures.

Reproducibility of the *real* build rests on three properties:

1. **Deterministic hashing.** MinHash shingles are hashed with `xxhash` rather than Python's salted builtin. The practical evidence: two independent full Nepali cascades produced byte-identical stage counts (286,181 survivors; 43,378 recurring paragraphs; 1,836,622 paragraphs removed).
2. **Deterministic splits.** Held-out membership derives from a hash of the document itself, with the absolute cap applied by stream position, so repeated passes agree without sharing state and a rerun reproduces the same splits regardless of shard iteration order.
3. **Resumability.** `--from-interim` redoes only the split and decontamination phase from a previous run's interim files, so changing the held-out size or the corpus budget costs minutes rather than repeating the ∼1.5-hour cascade. The resumed path and the full path call the same function, so they cannot drift apart.

The test suite is **354 tests**, including cross-process determinism of MinHash signatures, MinHash validated against the independent `datasketch` library, and tokenizer metrics validated against an independently trained HuggingFace BPE at the same vocabulary size.

9 Engineering Notes: Bugs Found by Running, Not Reading

Every item below was found by running the pipeline against real data. They are recorded because the diagnosis, in each case, is more instructive than the fix.

1. **The safety blocklist was deleting good prose.** It was built with `"...".split()`, which shredded multi-word entries into their component words — leaving the Hindi words for "of", "do" and "how" in the blocklist. Every ordinary article hits two of those, and the filter silently deleted **5.4 %** of perfectly good documents (157 of 2,922 in the measurement sample). It is now an explicit tuple, with single-token slurs and multi-word instructional phrases counted separately: a court report quoting a slur is legitimate prose, while an instruction has no innocent reading. A test caught a follow-on off-by-one where a document containing *two* explicit self-harm instructions was reported safe.
2. **Coherence scored zero for every document** — the IDF issue described earlier.
3. **A hand-picked threshold was wrong by an order of magnitude.** The coherence cutoff was set to 0.04 by eye; measurement placed that at Wikipedia's 10th percentile.
4. **JSON metadata was only stripped from the start of a document.** The blobs sit mid-document and are nested, so 1,285 documents were rejected for a symbol-to-word ratio caused entirely by punctuation the pipeline believed it had already removed. A balanced-brace scanner replaced the regex, which could not match nesting.
5. **Absolute thresholds that could never fire.** The ad-heavy-cluster threshold was 0.35, while the measured maximum cluster ad-rate is 0.0214 — it was structurally incapable of firing. Replaced with a relative rule (95th percentile plus a 2×-median guard), which flags 3 of 60 clusters in both languages.
6. **A threshold that removed real prose.** The duplicate-5-gram limit of 0.30 was rejecting legitimate Hindi news — gold prices quoted city by city, IPO listings, state-by-state weather. Reading the rejected documents (Lecture 3's explicit advice) and measuring the distributions moved it to 0.40, which recovers that prose while admitting almost no junk, since the crawl distribution is bimodal.

7. **Three memory-scaling failures on a 7.4 GB machine.** The cascade held the whole corpus in memory (fixed by streaming through gzipped interim files, with worker count capped by *available memory* rather than core count); SentencePiece died on a 5M-sentence sample (cut to 1M, stride-sampled); and the encoder accumulated every document's array before concatenating (∼2.3 GB peak, fixed by streaming writes).

8. **Splits silently desynchronised from their manifests.** The split writer sanitised only the newline character, but Python's universal-newline reading also splits on carriage return, and the downloaded side is stored raw by design. **21,823 Hindi documents** were written as one line and read back as two, misattributing the source of every document after the first occurrence. This was caught by a lockstep length check added while rewriting the statistics function; without it, the corruption would have flowed straight into the reported manual-versus-downloaded token split.

10 Deliverables and Artifact Locations

#	Deliverable	Location
1	Dataset collection scripts	`scripts/run_crawl.py`, `fast_crawl.py`, `run_wiki.py`, `fetch_downloaded.py`; `lma/crawl/`
2	Preprocessing pipelines	`hindi/clean/` and `nepali/clean/` (10 modules each), dispatched by `scripts/build_corpus.py`
3	Dataset statistics	`<lang>/data/stats.json`, `result.md`, this report, `report/figures/`
4	Train/val/test splits	`<lang>/data/splits/` — hosted on Drive and HuggingFace (5.5 + 7.5 = 13.0 GB; too large for the repository)
5	Tokenizer training code	`lma/tokenizer/` (`train_sp.py`, `sample.py`, `analyze.py`, `encode_corpus.py`), `scripts/tokenizer_sweep.py`
6	Vocabulary files	`hindi/tokenizer/hindi_unigram_16000.vocab`, `nepali/tokenizer/nepali_unigram_16000.vocab`
7	Tokenizer model files	`hindi_unigram_16000.model`, `nepali_unigram_16000.model`

Where the data lives. The corpus artifacts total 19.9 GB (essential) or 25.7 GB including regenerable intermediates, so they are hosted externally and are excluded from the git repository by `.gitignore`:

Path	Size	Why it is not in the repository
`<lang>/data/raw/`	3.1 + 3.8 GB	Irreplaceable — re-crawling returns different pages
`<lang>/data/splits/`	5.5 + 7.5 GB	The corpus itself; regenerable from raw + interim in ∼40 min per language
`<lang>/data/encoded/`	1.0 + 1.2 GB	Phase-2 training input; regenerable in ∼15 min
`<lang>/data/interim/`	1.2 + 1.9 GB	Optional; enables `--from-interim` re-splitting

The selected tokenizer models and vocabularies (∼0.7 MB each) *are* committed, as are all statistics files, so every number in this report can be checked against the repository without downloading the corpus.

11 Limitations and Remaining Work

Stated honestly, since each is a real qualification on the results above:

1. **Held-out sets are mildly biased** by the decision to decontaminate the held-out side rather than the training side (Section 4.3). Validation and test documents are, by construction, ones that do not share 13-grams with training text.
2. **The downloaded side is not independently verified.** IndicCorpV2 is trusted to be cleaned; this project deduplicated it against the manual side but applied no quality filtering to it, by design. Its internal quality is therefore inherited, not measured.
3. **The parameter-budget analysis rests on two assumptions** ($d_{\text{model}} = 512$, tied embeddings) that Phase 2 has not yet fixed. The fertility measurements are unaffected; only the affordability argument depends on them.
4. **Vocabulary size is defensibly chosen, not proven optimal.** 12k is a genuine alternative: it costs 25 % of the parameter budget rather than 33 %, has the highest vocabulary utilisation measured (0.978 Hindi / 0.975 Nepali), and gives up ∼3 % fertility. The final judgement properly belongs with Phase 2's architecture decision.

All numbers in this report are generated from `<lang>/data/stats.json`, `<lang>/tokenizer/sweep_results.json`, `<lang>/data/encoded/encoding.json` and `report/figures/langid_confusion.json`, which are produced by the pipeline itself and committed to the repository. `result.md` in the repository root is a regenerable side-by-side summary (`python scripts/summarize_runs.py`).